

1. Problem Understanding & Framing *Alternative Credit Scoring Proof-of-Concept for Rural Banks*

According to the Rural Bankers' Association of the Philippines, rural banks serve as critical "last-mile" financial channels that drive grassroots economic growth and financial inclusion. They are present in remote and lower-income municipalities and have the greatest potential impact on the local communities they serve. Among the Philippine rural banks most crucial functions are:

1. Advancing Financial Inclusion: Operating in municipalities deemed unviable by commercial banks, they bring basic services to unbanked populations.
2. Supporting Countryside Development: Channeling credit directly to small-scale farmers, fisherfolk, and micro-entrepreneurs.
3. Serving as Government Conduits: Acting as distribution points for public financial aid and agricultural credit programs.
4. Acting as Local Financial Mobilizers: Accepting local savings and deposits to reinvest back into the immediate rural community.

In terms of technology adoption, Rural banks are seen to lag behind their Universal, Commercial, Thrift and Digital banking counterparts. Rural banks more often than not, have the following limitations:

- Limited tech budgets
- Poor remote internet connectivity
- Acute shortages in IT talent
- Heavy reliance on legacy and manual systems

Because of this technology lag, smaller rural banks with fewer resources are not able to capitalize on recent financial inclusion breakthroughs such as the use of financial platform engagement, utility payment histories and mobile top-up habits in scoring unbanked applicants.

PROJECT OBJECTIVE: is to create a proof-of-concept alternative credit scoring algorithm for rural banks. This algorithm will take into account non-traditional parameters such as electronic money use, utility payments and mobile use. The goal is to provide a lightweight framework which can be customized very rapidly for a particular bank's dataset, processes and risk appetite.

Credit scoring is basically a **BINARY CLASSIFICATION** machine learning problem. The goal is to predict whether an applicant is a good borrower, or a default risk. What's peculiar about credit scoring is that we are more interested in the ability to correctly predict the MINORITY or decline/default class in order to safeguard or improve the lender's loans portfolio. The success metrics for this model is NOT the traditional overall Accuracy score, but the metrics for the MINORITY class:

- **PRECISION:** % of actual decline/default that were correct
- **RECALL:** % of actual decline/default predicted
- **F1 SCORE:** which balanced Precision & Recall in a single score

BUSINESS IMPACT include:

- from UPLIFT approval of loan applications that would have been rejected using traditional evaluation, and,
- loan PORTFOLIO IMPROVEMENT from rejection of applications that would have been approved using traditional parameters, but, rejected upon evaluation with additional data.

REFERENCES:

- [Rural Bankers Association Of The Philippines: 70 Years Of Touching Lives And Promoting Inclusive Development](#)
- [Advancing Financial Inclusion in the Philippines through Digital Platforms and AI Deployment](#)
- [Lenddo and CRIF partner to offer unique credit scoring solution to rural and savings banks in the Philippines](#)

2. Data Collection & Understanding

Loan application with approval and denial data is private, confidential and proprietary. Fortunately, World Bank has a dataset, *The Global Findex Database 2025: Connectivity and Financial Inclusion in the Digital Economy*, which could stand in for loan application information. Full documentation as well as data dictionary of this set can be found here: <https://microdata.worldbank.org/catalog/7860/get-microdata>.

DATASET SOURCE ATTRIBUTION: Development Research Group, Finance and Private Sector Development Unit (World Bank) - The Global Findex Database 2025: Connectivity and Financial Inclusion in the Digital Economy (FINDEX 2025). Downloaded from <https://microdata.worldbank.org/catalog/7860/get-microdata> on 22 September, 2026.

Initially, the Philippine subset of the Findex Database was targeted for use in this project. However, there are not enough data points (64) for loan application and approval in the Philippine subset.

To boost the number of samples, the Global superset of the Findex dataset will be used. This data collection has a total of 144,090 cases, 9,691 of which have loan application and approval or denial flags.

This number of datapoints is sufficient to train a classification model. However, use of a global database restricts immediate use of the resulting model in a local, Philippine setting. The model would have to be fine-tuned or re-trained with a bank's actual data prior deployment.

The World Bank dataset is comprehensive survey of indicators; it consists of 199 variables across different categories:

CATEGORY	FEATURES
Demographic / Psychographic	27
Socioeconomic - Traditional	68
Socioeconomic - Alternative	102
Others (Survey parameters)	2
TOTAL	199

Demographics information include country, population, age, and other classic data points Traditional Socioeconomic indicators includes such things as household income quintile, credit card ownership and use, and savings. Alternative indicators include utility payments, frequency of digital transactions and mobile phone and plan ownership.

Next challenge is to narrow down which among these variables are most impactful in our alternative scoring algorithm.

3. Data Preprocessing, Applied EDA & Feature Engineering

ROW FILTERING: out of the 144K+ data points, we will consider only the records where the respondent did apply for a loan (variable 70, "Applied for a loan using a mobile phone"). This cuts out 93% of the records, leaving us with a much more manageable 9,691-record dataset. In addition, remove any duplicates.

```
import pandas as pd

df = pd.read_csv('index_microdata_2025_labelled_update112425.csv') # Load the dataset

df = df[~df['fin20'].isin([2, 3, 4, pd.NA])] # remove records where respondent did not apply for a loan
df.dropna(subset=['fin20'], inplace=True)    # remove records with null data in loan application question

print(f"DataFrame shape after filtering: {df.shape}")

df.to_csv('index_microdata_2025_loans.csv', index=False) # create offline copy of the filtered dataset for future reprocessing
```

```
print(f"Number of duplicate rows in df: {df.duplicated().sum()}")
```

Number of duplicate rows in df: 0

NOTE: there are no duplicates in our dataset

FEATURE SELECTION: given we have 9.7K datapoints with an imbalanced dataset (74:26 ratio of approved loans vs declined loans; see database documentation [ddi-documentation-english_microdata-7860.pdf](#)) we will use the Events Per Variable (EPV) rule of thumb to target the number of features in our model:

- $9,691 \times 26.2\%$ declined rate = 2,537 minority class
- $2,537 \text{ minority class} / 15 \text{ to } 20 \text{ EPV} = 132 \text{ to } 169$ maximum features

NOTE that this is a MAX figure, a robust model could be created with fewer than this maximum number.

FEATURE SELECTION STEPS:

1. Remove **prohibited attributes**: demographics or socioeconomic indicators that could perpetuate bias such as race, color, religion, national origin, sex, marital status & age. SOURCE: US Equal Credit Opportunity Act (ECOA). Also drop features that are Findex survey constants (e.g. Year, Applied for Loan Indicator). Add to dropped columns irrelevant attributes such as occurrences of calamity. See FeatureSelection.pdf for list and rationale for excluding these columns.
2. Review of the dataset documentation reveal a large number of variables are sparsely populated. **Drop columns with > 50% NULL values** to avoid adding noise and non-organic bias to the dataset, and, to simplify training.
3. Perform **domain grouping** on the features with values that are insignificant (less than 5%) to control number of dummy variables once we do one-hot encoding.
4. Use **Chi-square test of association** to determine if each of the remaining attributes are statistically relevant to our target variable

3. Data Preprocessing, Applied EDA & Feature Engineering (cont'd)

#1 Drop COA, constants and other irrelevant columns

```
columns_to_drop = ["year", "economy", "economycode", "regionwb", "pop_adult", "female", "age", "fin15", "fin20", "fin24c", "fin24d1", "fin24d2", "fin24d3", "fin25e4", "con15", "fin49a", "fin49b", "fin49c", "fin49d"]
df = df.drop(columns=columns_to_drop, errors='ignore')
print(f"DataFrame shape after dropping columns: {df.shape}")
```

DataFrame shape after dropping columns: (9691, 178)

#2 Check for sparsity

```
import pandas as pd
```

```
null_counts = df.isnull().sum()
```

```
columns_with_nulls = null_counts[null_counts > 0].sort_values(ascending=False)
```

```
if not columns_with_nulls.empty:
```

```
    print("Columns with null values and their percentages:")
```

```
    null_percentages = (columns_with_nulls / len(df)) * 100 # Calculate percentage of nulls
```

```
    high_null_columns = null_percentages[null_percentages >= 50] # Filter for columns with >= 50% null values
```

```
    if not high_null_columns.empty:
```

```
        print("Columns with >= 50% null values and their percentages:")
```

```
        pd.set_option('display.max_rows', None) # Set option to display all rows
```

```
        print(high_null_columns.apply(lambda x: f"{x:.2f}%"))
```

```
        pd.reset_option('display.max_rows')
```

```
        print(f"\nTotal columns with >= 50% null values: {len(high_null_columns)}")
```

```
    else:
```

```
        print("No columns found with 50% or more null values.")
```

```
    print(f"\nTotal columns in DataFrame: {df.shape[1]}")
```

```
else:
```

```
    print("No null values found in any of the remaining columns.")
```

Drop extremely sparse attributes

```
if not high_null_columns.empty:
```

```
    df = df.drop(columns=high_null_columns.index)
```

```
    print(f"\nDropped {len(high_null_columns)} columns with >= 50% null values.")
```

```
    print(f"DataFrame shape after dropping sparse columns: {df.shape}")
```

```
else:
```

```
    print("\nNo columns to drop based on the 50% null value threshold.")
```

3. Data Preprocessing, Applied EDA & Feature Engineering (cont'd)

```
#2b: imputation (cont'd): fill in the missing NULL values
# use supermajority value where available in the other sets; e.g. if "YES" > 2/3 of values, then use "YES" as the default value
# where there is no clear pattern in the data, use the equivalent of "I DON'T KNOW" so as not to introduce any unnatural patterns in the data
# see "rationale column" in 'findex_microdata_2025_loans-ImputationDefaults.csv' for details
imputation_defaults_df = pd.read_csv('findex_microdata_2025_loans-ImputationDefaults.csv')
print("Loaded 'findex_microdata_2025_loans-ImputationDefaults.csv' successfully.")
display(imputation_defaults_df.head())
```

```
for index, row in imputation_defaults_df.iterrows(): # Impute NULL values in df
    column_name = row['Column']
    default_value = row['default']
    if column_name in df.columns:
        df[column_name] = df[column_name].fillna(default_value)
        print(f"Imputed column '{column_name}' with default value '{default_value}'.")
    else:
        print(f"Warning: Column '{column_name}' not found in df. Skipping imputation for this column.")
```

```
#3 domain grouping (cont'd): after inspecting the rare values, data dict was used to decide which values to combine
# disposition is in rare_value_replacement.csv
import pandas as pd
```

```
rare_value_replacement_df = pd.read_csv('rare_value_replacement.csv')
print(f"Loaded 'rare_value_replacement.csv'. Shape: {rare_value_replacement_df.shape}")
display(rare_value_replacement_df.head())
```

```
# Replace rare values with replacement values based on the mapping
replacements_made = 0
for index, row in rare_value_replacement_df.iterrows():
    col = row['Column']
    rare_val = row['Rare_Value']
    repl_val = row['Replacement_Value']

    if col in df.columns:
        df[col] = df[col].replace(rare_val, repl_val)
        replacements_made += 1
```

```
print(f"\nSuccessfully applied {replacements_made} replacement rules to df.")
```

3. Data Preprocessing, Applied EDA & Feature Engineering (cont'd)

```
#4 Chi-Square test of association; target p-value of 0.25 or less
import pandas as pd
from scipy.stats import chi2_contingency

dependent_variable = 'fin21' # Dependent variable
columns_to_exclude = ['wpid_random', 'wgt', dependent_variable] # Columns to exclude from independent variables
independent_variables = [col for col in df.columns if col not in columns_to_exclude] # Get list of independent variables (all columns except excluded ones)

print(f"Performing Chi-square test of association against dependent variable: '{dependent_variable}'\n")

chi_square_results = []

for col in independent_variables:
    contingency_table = pd.crosstab(df[col], df[dependent_variable]) # Create a contingency table
    chi2, p_value, dof, expected = chi2_contingency(contingency_table) # Perform Chi-square test
    min_expected_frequency = expected.min() # Check for minimum expected frequency
    min_expected_check = True if min_expected_frequency >= 5 else False

    result = {
        'Independent Variable': col,
        'Chi2 Statistic': chi2,
        'P-value': p_value,
        'Degrees of Freedom': dof,
        'Min Expected Frequency': min_expected_frequency,
        'Min Expected Frequency Met (>=5)': min_expected_check
    }
    chi_square_results.append(result)

    print(f"--- Results for '{col}' vs '{dependent_variable}' ---")
    print(f"Chi-square Statistic: {chi2:.4f}")
    print(f"P-value: {p_value:.4f}")
    print(f"Degrees of Freedom: {dof}")
    print(f"Minimum Expected Frequency: {min_expected_frequency:.2f}")
    if not min_expected_check:
        print("WARNING: Minimum expected frequency is less than 5. Chi-square test results may be unreliable.")
    print("\n")

# display all results in a DataFrame
chi_square_df = pd.DataFrame(chi_square_results)
print("Summary of Chi-square Tests:")
display(chi_square_df.sort_values(by='P-value'))

# Suggest columns to keep and drop based on p-value threshold
p_value_threshold = 0.25
columns_to_keep = chi_square_df[chi_square_df['P-value'] < p_value_threshold]['Independent Variable'].tolist()
columns_to_drop_suggested = chi_square_df[chi_square_df['P-value'] >= p_value_threshold]['Independent Variable'].tolist()

print(f"\n--- Feature Selection Suggestion (P-value < {p_value_threshold}) ---")
print(f"Suggested columns to KEEP (P-value < 0.25):")
print(columns_to_keep)
print(f"\nSuggested columns to DROP (P-value >= 0.25):")
print(columns_to_drop_suggested)
```

EDA & Feature Selection & Engineering Summary:

- The WB Findex Database rows was filtered select only datapoints with loan application (variable fin20=YES)
- Attributes set was trimmed:

FEATURE SET	COUNT
BEGINNING FEATURE SET	199
LESS: Prohibited, Constant, Ambiguous/Irrelevant	(21)
LESS: very sparse (> 50% NULL)	(91)
LESS: Statistically Irrelevant (Failed χ^2 Association Test)	(6)
FINAL FEATURE SET	81

- The attributes among this 81 with remaining NULL values were imputed with categorical values using predominantly the "Don't know" option, or a Supermajority value where available; see *findex_microdata_2025_loans-ImputationDefaults.csv* for the imputed values per field and the rationale for each feature
- To further minimize noise, the remaining attributes with values that comprise 5% for that value (rare values) were grouped with other values (mostly "don't know"); see *rare_value_replacement.csv* for the values replaced, the replacement value, and the rationale for choosing the replacement value for each attribute

The **data dictionary** for the dataset to be used for model training is in

JOSEPH_HANS_Pillar_5_Capstone-DataDictionary - FinalAttributesUsedInCapstone.csv

4. Model Implementation

Trial 3 models for our alternative credit scoring:

- Lasso with Logistic Regression
- CatBoost
- XGBoost

```
# create train-test split from our cleaned-up dataset
from sklearn.model_selection import train_test_split

y = df['fin21']                # Define target variable (y)
X = df.drop(columns=['fin21', 'wpid_random', 'wgt']) # features (X)
wgt = df['wgt']                # weights (wgt)

X_train, X_test, y_train, y_test, wgt_train, wgt_test = train_test_split(
    X, y, wgt, test_size=0.2, random_state=42, stratify=y
)
```

4A Lasso Logistic Regression

```
# A2. Lasso Logistic Regression with Balanced Class Weights
from sklearn.model_selection import GridSearchCV
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix, classification_report

# Define the preprocessing and modeling pipeline
preprocessor = OneHotEncoder(handle_unknown='ignore')

# Initialize Logistic Regression with balanced class weights
log_reg = LogisticRegression(penalty='l1', solver='liblinear', class_weight='balanced', random_state=42, max_iter=1000)

pipeline_cv = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('classifier', log_reg)
])

# Define a grid of hyperparameters to search
param_grid = {
    'classifier__C': [0.01, 0.1, 1, 10, 100]
}

print("Starting Grid Search for Logistic Regression...")
# We use cv=3 to match the previous GridSearch
grid_search_lr = GridSearchCV(estimator=pipeline_cv, param_grid=param_grid, scoring='accuracy', cv=3, n_jobs=-1)
grid_search_lr.fit(X_train, y_train)

print(f"Best Parameters: {grid_search_lr.best_params_}")

# Use the best estimator to predict on the test set
best_lr_model = grid_search_lr.best_estimator_
y_pred_best_lr = best_lr_model.predict(X_test)

# Evaluate the improved model
acc_best_lr = accuracy_score(y_test, y_pred_best_lr)
prec_best_lr = precision_score(y_test, y_pred_best_lr)
rec_best_lr = recall_score(y_test, y_pred_best_lr)
f1_best_lr = f1_score(y_test, y_pred_best_lr)
conf_matrix_best_lr = confusion_matrix(y_test, y_pred_best_lr)
```

4B CatBoost

```
# B. CatBoost
!pip install -q catboost
```

```
# B1. Catboost with balanced class weights
from catboost import CatBoostClassifier
from sklearn.model_selection import GridSearchCV
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix

# Initialize CatBoost with balanced class weights to handle the 74:26 imbalance
cat_base = CatBoostClassifier(random_state=42, verbose=0, auto_class_weights='Balanced')

# Define a grid of hyperparameters to search
param_grid = {
    'iterations': [100, 200],
    'depth': [4, 6, 8],
    'learning_rate': [0.01, 0.05, 0.1]
}

print("Starting Grid Search for CatBoost...")
# We use cv=3 to keep training time manageable in this notebook
grid_search = GridSearchCV(estimator=cat_base, param_grid=param_grid, scoring='accuracy', cv=3, n_jobs=-1)
grid_search.fit(X_train, y_train)

print(f"Best Parameters: {grid_search.best_params_}")

# Use the best estimator to predict on the test set
best_cat_model = grid_search.best_estimator_
y_pred_best_cat = best_cat_model.predict(X_test)

# Evaluate the improved model
acc_best = accuracy_score(y_test, y_pred_best_cat)
prec_best = precision_score(y_test, y_pred_best_cat)
rec_best = recall_score(y_test, y_pred_best_cat)
f1_best = f1_score(y_test, y_pred_best_cat)
conf_matrix_best = confusion_matrix(y_test, y_pred_best_cat)
```

4C XGBoost

```
# C2. XGBoost option fine tuned
import xgboost as xgb
from sklearn.model_selection import GridSearchCV
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix, classification_report

# Calculate the ratio of negative (majority) to positive (minority) instances for scale_pos_weight
scale_pos_weight_value = sum(y_train_xgb == 0) / sum(y_train_xgb == 1)
print(f"Calculated scale_pos_weight: {scale_pos_weight_value:.4f}")

# Initialize XGBoost with the balanced weight
xgb_base = xgb.XGBClassifier(random_state=42, eval_metric='logloss', scale_pos_weight=scale_pos_weight_value)

# Define a grid of hyperparameters to search
param_grid_xgb = {
    'max_depth': [3, 5, 7],
    'learning_rate': [0.01, 0.05, 0.1],
    'n_estimators': [100, 200]
}

print("Starting Grid Search for XGBoost...")
# Use cv=3 to keep training time manageable
grid_search_xgb = GridSearchCV(estimator=xgb_base, param_grid=param_grid_xgb, scoring='accuracy', cv=3, n_jobs=-1)
grid_search_xgb.fit(X_train, y_train_xgb)

print(f"Best Parameters: {grid_search_xgb.best_params_}")

# Use the best estimator to predict on the test set
best_xgb_model = grid_search_xgb.best_estimator_
y_pred_best_xgb_raw = best_xgb_model.predict(X_test)

# Shift predictions back by adding 1 to match y_test labels (1 and 2)
y_pred_best_xgb = y_pred_best_xgb_raw + 1

# Evaluate the improved model
acc_best_xgb = accuracy_score(y_test, y_pred_best_xgb)
prec_best_xgb = precision_score(y_test, y_pred_best_xgb)
rec_best_xgb = recall_score(y_test, y_pred_best_xgb)
f1_best_xgb = f1_score(y_test, y_pred_best_xgb)
conf_matrix_best_xgb = confusion_matrix(y_test, y_pred_best_xgb)
```

	Minority Precision	Minority Recall	Minority F1-Score
Model			
Tuned Lasso LR	0.4197	0.6922	0.5226
Tuned CatBoost	0.4715	0.5020	0.4862
Tuned XGBoost	0.4910	0.4824	0.4866

MODEL EVALUATION: of the three tuned models, it is **Lasso Linear Regression** that has the best RECALL of the default/decline class. It's PRECISION is lowest, so there will be more "approve" cases that are misclassified as "decline". This makes the model more **conservative**, which is preferable, especially for small lenders who have limited resources, or, for banks that are just starting with alternative scoring and want to be careful in safeguarding their loan portfolio at the initial stages.

The **Tuned Lasso LR is recommended for alternative credit scoring** for Rural Banks.

5. Model Audit

5.i SHAP feature importance

```
#5.i SHAP feature importance
import shap
import matplotlib.pyplot as plt
# import pandas as pd # pandas is already imported in 0caab115 and is available
import numpy as np # Import numpy

# Extract the OneHotEncoder and LogisticRegression model from the pipeline
ohe = best_lr_model.named_steps['preprocessor']
classifier = best_lr_model.named_steps['classifier']

# Transform X_train to get feature names for SHAP
X_train_encoded = ohe.transform(X_train)
feature_names = ohe.get_feature_names_out(X.columns)

# Convert to DataFrame for easier handling with SHAP
X_train_encoded_df = pd.DataFrame(X_train_encoded.toarray(), columns=feature_names)

# Apply the mapping to get human-readable feature names
human_readable_feature_names = map_encoded_feature_names_to_labels(feature_names, column_label_map)

# Create a SHAP explainer for the Logistic Regression model
explainer = shap.LinearExplainer(classifier, shap.maskers.Independent(X_train_encoded_df, max_samples=X_train_encoded_df.shape[0]))

# Calculate SHAP values for the test set (after encoding)
X_test_encoded = ohe.transform(X_test)
X_test_encoded_df = pd.DataFrame(X_test_encoded.toarray(), columns=feature_names)

# Rename columns of X_test_encoded_df to human-readable names for SHAP plots
X_test_encoded_df.columns = human_readable_feature_names

# For binary classification, shap_values is a list of two arrays. We take the second array (index 1) for the positive class.
shap_values = explainer.shap_values(X_test_encoded_df)

# Visualize the feature importance
print("Generating SHAP summary plot...")
plt.figure(figsize=(20, 120)) # Significantly increased figure height for many features
shap.summary_plot(shap_values, X_test_encoded_df, plot_type="bar", show=False)
plt.title("SHAP Feature Importance for Lasso Logistic Regression")
ax_bar_plot = plt.gca() # Get the current axes object that SHAP created for the bar plot
mean_abs_shap_values_for_plot = np.mean(np.abs(shap_values[1]), axis=0) # Calculate the mean absolute SHAP values for annotation, sp

# shap.summary_plot for bar plot type ranks features by mean absolute SHAP value (descending) # Map these values to feature names and
feature_importance_df = pd.DataFrame({
    'feature': human_readable_feature_names,
    'importance': mean_abs_shap_values_for_plot
}).sort_values(by='importance', ascending=False).reset_index(drop=True)
plt.tight_layout()
plt.show()
```

5.ii L1 Penalty in Lasso used in explainability

```
#5.ii L1 Penalty in Lasso as indicator of feature importance
import pandas as pd
import re # This import is needed for the regex in the function

# Load the column labels from the data dictionary
column_labels_df = pd.read_csv('JOSEPH_HANS_Pillar_5_Capstone-DataDictionary-ColumnLabels.csv')
# Create a dictionary for easy lookup: original_column_name -> Label
column_label_map = dict(zip(column_labels_df['ColumnName'], column_labels_df['Label']))

def map_encoded_feature_names_to_labels(encoded_feature_names, label_map):
    new_names = []
    for encoded_name in encoded_feature_names:
        match = re.match(r'^(.*?)_([0-9\+])$', encoded_name)
        if match:
            base_name, value = match.groups()
            if base_name in label_map:
                new_names.append(f"{label_map[base_name]}_{value}")
            else:
                new_names.append(encoded_name)
        else:
            if encoded_name in label_map:
                new_names.append(label_map[encoded_name])
            else:
                new_names.append(encoded_name)
    return new_names

# Access the trained Logistic Regression model from the pipeline
logistic_reg_model = best_lr_model.named_steps['classifier']

# Get the feature names after OneHotEncoding
ohe = best_lr_model.named_steps['preprocessor']
encoded_feature_names = ohe.get_feature_names_out(X.columns)

# Get the coefficients (L1 penalty) from the logistic regression model
# For binary classification, coef_ is usually shape (1, n_features), so we take the first row.
coefficients = logistic_reg_model.coef_[0]

# Create a DataFrame for feature importance
feature_importance_l1_df = pd.DataFrame({
    'feature': encoded_feature_names,
    'coefficient': coefficients
})

# Map the encoded feature names to human-readable labels using the already defined function
feature_importance_l1_df['human_readable_feature'] = map_encoded_feature_names_to_labels(
    feature_importance_l1_df['feature'], column_label_map
)
```

5. Model Audit (cont'd)

The L1 penalty in Lasso Logistic Regression shrinks the coefficients of less important features towards zero. Features with larger absolute coefficients are considered more important in the model's decision-making process. These top features, as indicated by their L1 coefficients, align with the SHAP results, highlighting the importance of prior borrowing experiences and financial behaviors.

5.iii. Model limitations

One major limitation of this model is the **global nature of the dataset**. It is not specific to the Philippine context and not specific to rural bank lending. To address this, the model needs to be **fine-tuned and/or retrained** with a bank's own data.

Gemini also suggests that the **linearity assumption** of LR is a potential limitation: it is assumed that the features have a linear relationship to the log-odds of the approve/decline target variable. If the relationships are complex and non-linear, then LR will not be able to capture this relationship. This potential limitation was addressed by exploring non-linear CatBoost and XGBoost alternatives, which yielded lower minority-class recall and F1 scores.

5.iv Bias & Fairness

- The usual **causes for bias** (gender, age, ethnicity using country of origin as proxy) **were eliminated** early in feature selection as these are prohibited for use in credit scoring algorithms.
- The important features that emerged during model training, though, seemed to **discriminate against the unbanked population**. The most important features were related to either a) prior financial experience with loans and b) capacity and motivation to pay. The algorithm thus perpetuates credit to those with existing credit history. It does not help advance financial inclusivity.
- To address this, we can create an **alternative model** where we drop the traditional financial attributes so that the importance of alternative attributes (e.g. utility payments, rental payments) can emerge. This exercise is shown in the cells below.
- The recall of this alternative model is 0.5863, still better than the proportion of negative cases in our dataset. However, it is below that of best_lr_model which includes the traditional financial variables.
- To further improve this model, next step should be to expand dataset to include actual loan repayment data (versus the loan approval target variable that we are limited to currently).

5.iv.1 Addressing Bias: Focusing on Alternative Financial Trustworthiness Features

To address the potential bias against unbanked populations, we will modify our feature set by:

- **Excluding 'Traditional Financial Experience' Features:** These are features that often require prior engagement with formal banking institutions (e.g., having a bank account, borrowing from traditional banks, saving in formal institutions).
- **Emphasizing 'Alternative Financial Trustworthiness' Features:** These include indicators of financial behavior and discipline through digital transactions, mobile money, and consistent payments (e.g., paying utility bills, using mobile money, making digital merchant payments, managing larger purchases).

This approach aims to create a model that can assess creditworthiness based on broader and more inclusive financial behaviors, rather than solely on formal credit history, which many unbanked individuals lack.*

NOTE: contents truncated, for full details, see:

“JOSEPH_HANS_Pillar_5_Capstone-TechnicalDeck.html” or “JOSEPH_HANS_Pillar_5_Capstone-TechnicalDeck.ipynb”